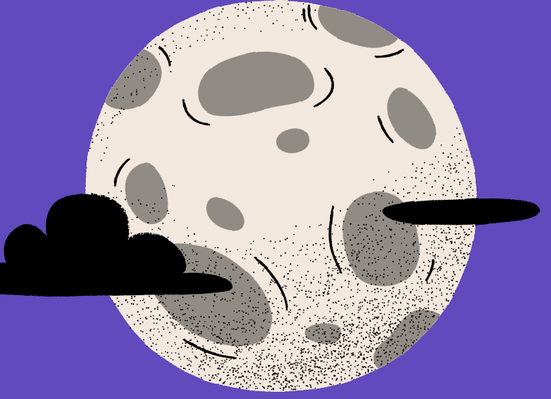
POO COM RUBY

VISÃO GERAL

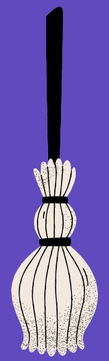
- Herança e Polimorfismo
- Encapsulamento

- Composição Vs Agregação
- Interfaces via Módulos



POO: OS 4 PILARES FUNDAMENTAIS

1. Abstração: Modelar conceitos do mundo real.
2. Encapsulamento: Proteger dados e controlar acesso.
3. Herança: Reutilizar e estender comportamentos.
4. Polimorfismo: Objetos diferentes respondendo ao mesmo método.



HERANÇA E POLIMORFISMO

- Herança: Usa < para estender outra classe.
- Polimorfismo (Duck Typing): Se algo "anda como um pato e faz quack", é um pato.
- Ruby liga o comportamento ao método, não ao tipo.

```ruby
class Veiculo
  def mover; puts "Movendo..."; end
end

class Carro < Veiculo
  def mover; puts "Carro acelerando!"; end
end

class Moto < Veiculo
  def mover; puts "Moto arrancando!"; end
end

[Carro.new, Moto.new].each(&:mover)
```

ENCAPSULAMENTO EM RUBY

Atributos não são acessíveis diretamente.
Acesso é feito via métodos (getters/setters).
Ruby facilita com:
attr_reader ➜ só leitura
attr_writer ➜ só escrita
attr_accessor ➜ leitura e escrita

```ruby
class Pessoa
  attr_accessor :nome

  def initialize(nome)
    @nome = nome
  end
end

p = Pessoa.new("Ana")
puts p.nome
p.nome = "Beatriz"
```

COMPOSIÇÃO VS. AGREGAÇÃO

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

🔸 Composição ➜ dependência total

🔹 Agregação ➜ ligação fraca (referência)

```ruby
# Composição
class Motor
  def ligar; puts "Motor ligado!"; end
end

class Carro
  def initialize
    @motor = Motor.new
  end
end

# Agregação
class Frota
  def initialize(veiculos)
    @veiculos = veiculos
  end
end
```

INTERFACES VIA MÓDULOS

- Ruby não tem interfaces formais.
- Modules definem contratos de comportamento.
- Classes podem “incluir” módulos com include.

```ruby
module Movimentavel
  def mover
    raise NotImplementedError, "Implementar em classe"
  end
end

class Robo
  include Movimentavel
  def mover; puts "Robo em movimento!"; end
end
```

EXERCÍCIO 1

Cenário: O Bar AfterLife em Night City serve drinks icônicos, cada um nomeado em homenagem a um mercenário lendário (tipo DrinkLenda) ou um drink comum (DrinkComum). O preço de um DrinkLenda é baseado no seu tempo de existência.

Tarefa:

1. Crie a classe base Drink com atributos encapsulados para _nome e _preco_base.
2. Implemente getters e setters controlados (@property e @setter) para o preço, garantindo que ele nunca seja negativo.
3. Crie a subclasse DrinkComum que herda de Drink.
4. Crie a subclasse DrinkLenda que herda de Drink e adiciona o atributo anos_desde_criacao.
5. Em DrinkLenda, sobrescreva o método que calcula o preço final, adicionando um bônus de $5 por ano de criação ao preco_base.
6. Implemente o método mágico __str__ em todas as classes. No main, peça ao usuário (via gets.chomp) para inserir o nome e o preço base de um drink e calcule o preço final de um DrinkLenda usando o método sobrescrito.

EXERCÍCIO 2

Cenário: Em uma orquestra, vários músicos (Pianista, Violinista, Maestro) precisam realizar suas ações para executar uma peça.

Tarefa:

1. Crie uma classe base Musico com atributos para nome e instrumento.
2. Crie um método tocar_partitura(peca) que deve ser implementado por todas as subclasses.
3. Crie as classes filhas Pianista e Violinista que herdam de Musico.
4. Sobrescreva o método tocar_partitura(peca) em cada músico para refletir o instrumento e a peça.
5. Crie a classe Maestro. O Maestro deve ser capaz de criar uma "lista de execução" (Array) contendo objetos de Pianista e Violinista.
6. Crie um método iniciar_ensaio(peca) na classe Maestro que usa Array.each para chamar o método tocar_partitura(peca) em todos os músicos da sua lista. O Maestro deve também ter um método mudar_foco que usa .map para alterar o estado interno dos músicos (ex: "Concentrado").

EXERCÍCIO 3

Cenário: O protagonista Okabe (usuário) interage com o TelefoneDeMicroondas (o todo), que compõe uma MicroondasUnidade (peça vital) e agrega mensagens enviadas (D-Mails).

Tarefa:

1. Crie a classe MicroondasUnidade (Composição) com um método aquecer_por(tempo).
2. Crie a classe DMail (Agregação) com atributos conteudo e hora_envio.
3. Crie a classe TelefoneDeMicroondas.
    - Composição: No __init__, ele deve instanciar e compor o objeto MicroondasUnidade.
    - Agregação: Ele deve agregar uma lista (Array[DMail]) de mensagens enviadas.
4. Crie um método enviar_dmail() no TelefoneDeMicroondas. Este método deve:
    - Pedir ao usuário o conteúdo do D-Mail.
    - Criar uma nova instância de DMail.
    - Chamar o método da MicroondasUnidade para "aquecer" (Composição).
    - Adicionar o novo DMail à lista agregada.
5. Implemente um método listar_dmails() no TelefoneDeMicroondas que usa métodos de iteração (como .select ou .find) para mostrar apenas D-Mails enviados após um determinado horário (entrada do usuário).

EXERCÍCIO 4

Cenário: No conflito entre detetives e criminosos, todos os participantes podem ser Rastreaveis e Perigosos, mas apenas os criminosos são Mestres.

Tarefa:

1. Crie um module Rastreavel com um método obter_localizacao(hora).
2. Crie um module Perigoso com um método calcular_risco.
3. Crie a classe base Participante (com nome).
4. Crie a classe Detetive que herde de Participante e inclua Rastreavel.
5. Crie a classe MestreDoCrime que herde de Participante e inclua Rastreavel E Perigoso.
6. Crie a classe Cenario (que representa o jogo), que deve agregar uma lista polimórfica (Array[Rastreavel]).
7. No main:
    - Peça ao usuário a localização de Sherlock e Moriarty (via gets.chomp).
    - Crie as instâncias e adicione-as à lista de Cenario.
    - Use o Array.select na lista do Cenario para encontrar todos os objetos que também incluem o módulo Perigoso (filtrando assim apenas os criminosos).
    - Chame o método obter_localizacao para todos na lista original (Polimorfismo via Módulo).